BACKEND ENGINEERING · HLD INTERVIEW PREP

# Load Balancing

## Algorithms — Complete Study Notes

L4 vs L7 · Round Robin · Weighted Round Robin · IP Hash
Least Connections · Weighted Least Conn · Least Response Time

TOPICS COVERED

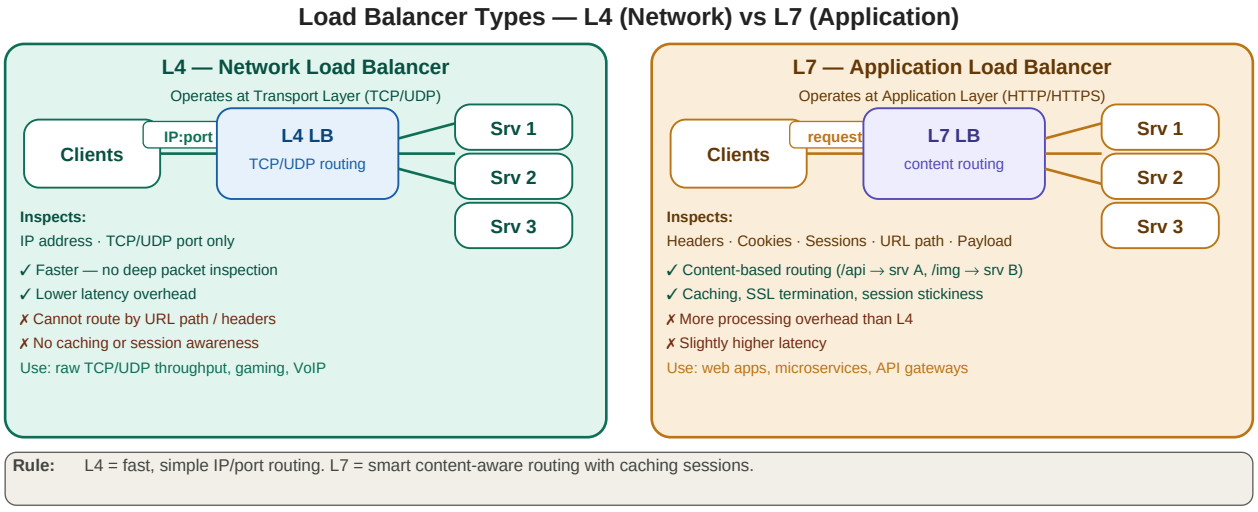
## Table of Contents

## 1. What is a Load Balancer?

A **load balancer** is a server or device that distributes incoming client requests across a pool of backend servers. Without it, a single server would receive all traffic — eventually becoming overwhelmed. The load balancer ensures no single server becomes a bottleneck, improving **availability, scalability, and fault tolerance**.

| Without Load Balancer | With Load Balancer |
|---|---|
| Single server handles all traffic | Traffic distributed across N servers |
| One server failure = complete downtime | One server failure = others absorb load |
| Cannot scale horizontally | Add servers behind LB to scale out |
| All users experience same slowdowns | LB routes to fastest/least-busy server |

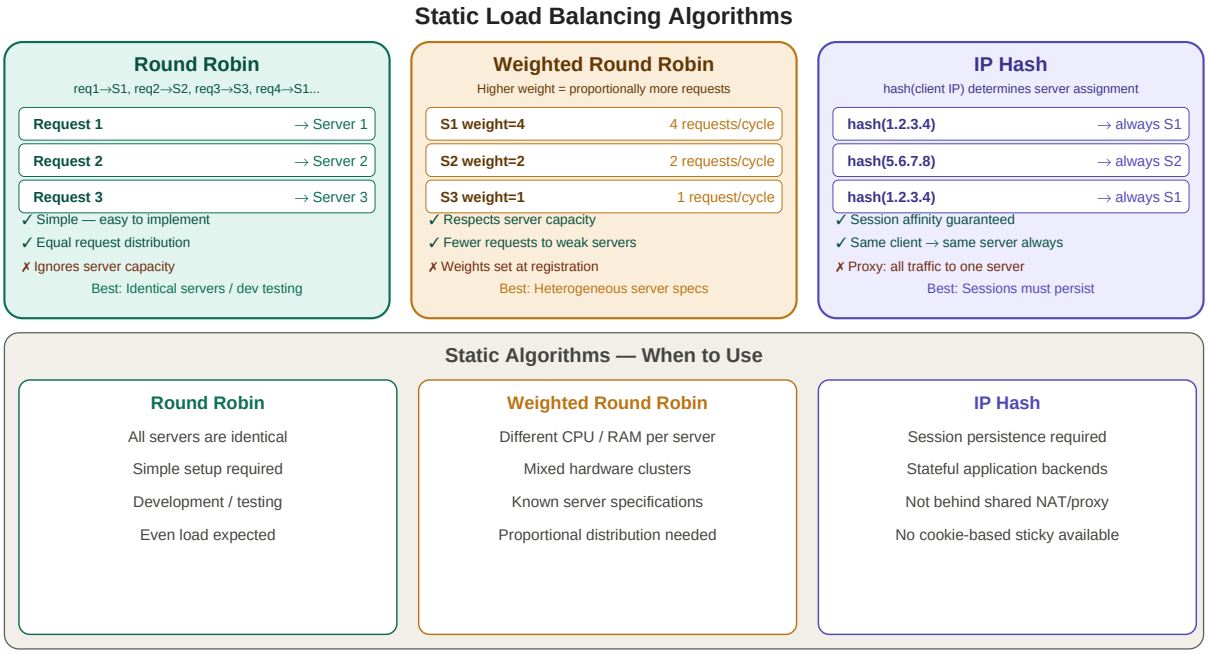
## 2. L4 vs L7 Load Balancers




L4: fast TCP/UDP routing by IP+port · L7: smart HTTP routing by headers/URL/cookies with caching & session support

## 3 & 4. Static Algorithms — Round Robin, Weighted RR, IP Hash

**Static algorithms** distribute load based on fixed, predetermined rules. They do not monitor current server state or adapt to real-time conditions. They are simpler to implement but less adaptive.




Round Robin: cyclic · Weighted RR: by capacity · IP Hash: session affinity · When-to-use cards below each

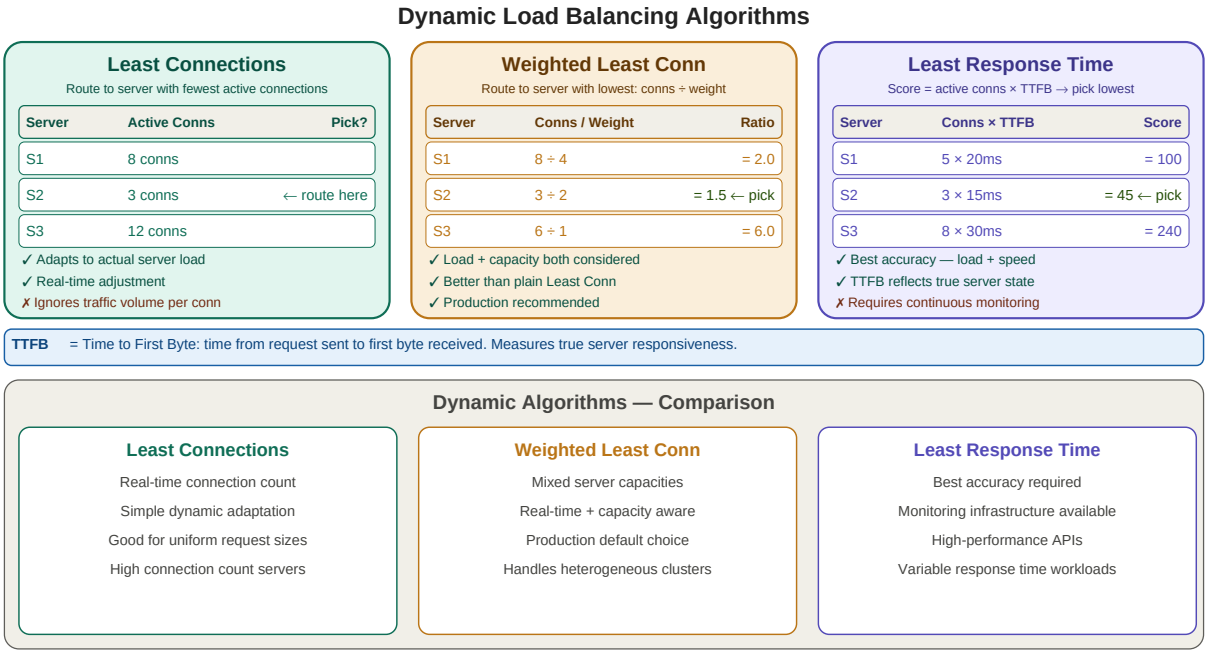
Round Robin: req 1→S1, req 2→S2, req 3→S3, req 4→S1... Works perfectly when all servers are identical.

Weighted Round Robin: S1(w=4) gets 4 requests, S2(w=2) gets 2, S3(w=1) gets 1 per cycle. Proportional to capacity.

IP Hash: hash(1.2.3.4) = 2 → S2 always. Same client always hits same server. Breaks if client goes via shared proxy.

## 5. Dynamic Algorithms — Real-Time Routing

**Dynamic algorithms** monitor the current state of each server and route requests based on live metrics. They adapt in real time, making them far better for production systems with variable load patterns.




Least Conn: min active connections · Weighted Least Conn: conns/weight ratio · Least Response Time: conns × TTFB score

### TTFB — Time to First Byte

**TTFB** measures the time from when a request is sent to when the first byte of the response is received. It is the best proxy for how busy a server currently is. A slow TTFB = server is processing heavy requests. Least Response Time uses TTFB to route to the genuinely fastest server right now.

| Algorithm | Formula | Route to | Example |
|---|---|---|---|
| Least Connections | count active conns | Server with min conns | S1=8, S2=3, S3=12 → route to S2 |
| Weighted Least Conn | conns ÷ weight | Server with min ratio | S1=8/4=2.0, S2=3/2=1.5, S3=6/1=6 → S2 |
| Least Response Time | conns × TTFB | Server with min score | S1=5×20=100, S2=3×15=45, S3=8×30=240 → S2 |

## 6. Algorithm Deep-Dive & Trade-offs

### Round Robin

- Each request goes to next server in rotation: S1, S2, S3, S1, S2, S3...
- No state tracking — LB does not know if servers are busy or free
- Problem: if S1 is 2x slower, it receives the same count as S2 and gets overwhelmed
- Fix: Use Weighted Round Robin when server specs differ

### Weighted Round Robin

- Assign weights proportional to server CPU/RAM capacity
- S1 (8 core, 32GB) → w=4, S2 (4 core, 16GB) → w=2, S3 (2 core, 8GB) → w=1
- Still static — if S1 gets 4 long-running requests it may still be slower than S2 with 2 quick ones
- Weights set at server registration time — do not update dynamically

### IP Hash

- Client IP hashed to a bucket → always same server. Useful for stateful sessions.
- If corporate users route via NAT/proxy, ALL share one IP → all requests go to one server
- Modern apps use sticky sessions via cookie instead (more reliable than IP hash)

### Least Connections

- LB tracks active connection count per server. New request → server with fewest.
- Problem: a server with 3 idle connections receives the same treatment as one with 3 active heavy queries
- Works well when requests are similar in duration (e.g., short API calls)

### Weighted Least Connections

- Ratio = active_connections / server_weight
- S1(8 conns, w=4) ratio=2.0 vs S2(3 conns, w=2) ratio=1.5 → route to S2
- Best of both worlds: real-time load AND server capacity considered
- Most production environments use this as their default dynamic algorithm

### Least Response Time

- Score = active_connections × TTFB. Lower score = route there.
- TTFB automatically captures both how busy the server is AND how fast it processes requests
- If two servers tie on score, fallback to Round Robin between them
- Requires constant monitoring — adds ~1ms overhead per health check cycle
- Best for heterogeneous workloads where response time varies significantly

## 7. Complete Algorithm Comparison

**All Algorithms — Quick Reference**

| Algorithm | Type | Metric | Pros | Cons | Best when |
|---|---|---|---|---|---|
| Round Robin | Static | Cyclic sequential | Simple, equal dist. | Ignores capacity complexity | identical servers |
| Weighted Round Robin | Static | Cyclic with weights | Handles capacity diff. | Ignores request processing time | mixed capacity |
| IP Hash | Static | hash(client IP) | Session affinity | Proxy = all traffic one server | session persist |
| Least Connections | Dynamic | Min active connections | Real-time adaptation | Ignores traffic volume/conn | high conn variety |
| Weighted Least Conn | Dynamic | conns / weight ratio | Load + capacity aware | Weights set manually | mixed + capacity |
| Least Response Time | Dynamic | conns × TTFB score | Most accurate routing | Monitoring overhead | perf-critical |

6 algorithms across: type, metric, pros, cons, best when

## 8. Quick Revision & Interview Questions

### Quick Revision

-> Load balancer = distributes requests across servers. Prevents single server bottleneck.

-> L4 LB: routes by IP+port at Transport Layer. Fast, simple, no content inspection.

-> L7 LB: routes by headers/URL/cookies at Application Layer. Smart, caching, session-aware.

-> Static algorithms: do not monitor server state. Fixed rules. Simpler but less adaptive.

-> Dynamic algorithms: monitor real-time metrics. More accurate but more complex.

-> Round Robin: cyclic. Equal distribution. No capacity awareness. Best for identical servers.

-> Weighted Round Robin: capacity weights. S1(w=4) gets 4x more than S3(w=1).

-> IP Hash: hash(IP) → same server. Session affinity. Breaks behind shared proxy/NAT.

-> Least Connections: route to server with fewest active connections. Real-time.

-> Weighted Least Conn: route to min(conns/weight). Best for mixed-capacity + real-time.

-> Least Response Time: score = conns × TTFB. Route to lowest score. Most accurate.

-> TTFB = Time to First Byte. Measures server responsiveness. Used in Least Response Time.

-> Tie in Least Response Time → fall back to Round Robin between tied servers.

-> Production default: Weighted Least Connections (real-time + capacity aware).

-> Session persistence via IP Hash is risky — prefer sticky cookies for stateful apps.

### Interview Questions

- 1. What is a load balancer and why is it needed?
- 2. What is the difference between an L4 and L7 load balancer?
- 3. Explain Round Robin. When does it break down?
- 4. What problem does Weighted Round Robin solve over plain Round Robin?
- 5. Why can IP Hash cause uneven load? How would you fix it?
- 6. Explain Least Connections. What is its limitation?
- 7. How does Weighted Least Connections work? Give a worked example.
- 8. What is TTFB? How is it used in Least Response Time?
- 9. When would you use a static algorithm vs a dynamic one?
- 10. If you had servers with different CPU/RAM specs, which algorithm would you pick?

- 11. How does a load balancer support high availability? What happens when one server fails?
- 12. Design a load balancing strategy for a payment API serving 1M requests/day.

## Key Terms

| | |
|---|---|
| **Load Balancer** | Distributes incoming requests across multiple servers to prevent overload |
| **L4 LB** | Network LB — routes by IP + TCP/UDP port. Fast, no content inspection. |
| **L7 LB** | Application LB — routes by headers, URL, cookies. Smart, supports caching. |
| **Round Robin** | Cyclic request distribution: S1, S2, S3, S1... No capacity awareness. |
| **Weighted RR** | Round Robin with capacity weights. Higher weight = more requests. |
| **IP Hash** | hash(client IP) → always same server. Session affinity. Proxy-unsafe. |
| **Least Connections** | Route to server with fewest active connections. Dynamic. |
| **Weighted Least Conn** | Route to min(conns/weight). Dynamic + capacity aware. |
| **Least Response Time** | Score = conns × TTFB. Route to lowest score. Most accurate. |
| **TTFB** | Time to First Byte — interval from request sent to first byte received. |
| **Static Algorithm** | Uses fixed rules, no real-time server monitoring. |
| **Dynamic Algorithm** | Monitors live server state and adapts routing in real time. |
| **Session Affinity** | Same client always routed to same server (sticky sessions). |
| **Health Check** | LB periodically checks if servers are alive and responsive. |